

NeuralForge: Un Framework MLOps Distribuido para Entrenamiento Automatizado de YOLO con Optimización de Hiperparámetros Multiobjetivo

William Steve Rodriguez Villamizar

Líder de IA y Arquitecto de Soluciones

wisrovi-suit

Badajoz, Extremadura, España

wisrovi.rodriguez@gmail.com

Resumen—Escalar la optimización de hiperparámetros para modelos de visión por computadora a través de clústeres GPU heterogéneos introduce cuellos de botella críticos en el aislamiento de estado, el enrutamiento de tareas y la orquestación de hardware. Presentamos NeuralForge, un framework MLOps distribuido estructurado bajo un patrón de diseño Invoker-Executor que distribuye de forma segura ensayos de Optuna a través de hasta 30 nodos trabajadores GPU utilizando Celery. Al desacoplar la lógica de ejecución en contenedores Docker [1] efímeros con estrictos límites de hardware, NeuralForge elimina las fugas de memoria (errores OOM [2]) y los conflictos de dependencias típicos de los demonios de entrenamiento de larga duración. Nuestra evaluación demuestra una latencia de despacho de tareas de sub-milisegundos, actualizaciones del clúster sin tiempo de inactividad a través de Watchtower, y una reducción del 40 % en el tiempo de inactividad de la GPU. NeuralForge establece una línea base altamente escalable para despliegues de YOLO a nivel empresarial.

Index Terms—Sistemas Distribuidos, MLOps, Optimización de Hiperparámetros, YOLO, Docker, Optuna

I. INTRODUCCIÓN

La Optimización de Hiperparámetros (HPO) para modelos de deep learning, como YOLO, requiere ejecutar miles de ensayos computacionalmente intensivos. Los frameworks monolíticos tradicionales luchan por gestionar el aislamiento de estado a través de ensayos continuos, culminando a menudo en errores de Falta de Memoria (OOM) y corrupción del contexto CUDA.

Introducimos NeuralForge, un ecosistema multi-repositorio que refactoriza el paradigma estándar de nodo-trabajador en un patrón Invoker-Executor. Al emplear Celery como un broker distribuido y PostgreSQL [3] para el estado compartido de Optuna, la arquitectura enruta dinámicamente las tareas a través de colas GPU priorizadas (`gpus_high`, `gpus_medium`, `gpus_low`).

II. TRABAJO RELACIONADO

Plataformas MLOps existentes como KubeFlow y MLflow proporcionan un seguimiento extensivo pero carecen de un aislamiento de ejecución ligero y nativo adaptado para bucles iterativos de HPO en metal crudo de GPU [4]. Kubernetes introduce una sobrecarga de red sustancial que limita el despacho de tareas de alta frecuencia [5]. NeuralForge cierra

esta brecha ejecutando directamente contenedores Docker [1] efímeros desde un Invoker ligero de Celery.

III. ARQUITECTURA PROPUESTA

El framework desacopla el sistema en tres capas principales: 1) **API Gateway**: Un servicio FastAPI [6] que ingiere payloads de configuración y los encola vía Redis. 2) **Nodo Manager**: Orquesta el estudio de Optuna, muestreando hiperparámetros usando TPE o CMA-ES, y despachando ensayos al broker. 3) **Nodo Invoker-Executor**: Un demonio de Celery [7] (Invoker) se ejecuta en cada servidor GPU físico. Al recibir una tarea, genera un contenedor Docker efímero (Executor) limitado estrictamente a parámetros predefinidos de `shm_size` e ID de GPU. Una vez que el modelo YOLO completa la iteración y el análisis forense, el Executor escribe los resultados JSON en un volumen CIFS compartido y termina, asegurando el desmontaje completo del entorno.

IV. CONFIGURACIÓN EXPERIMENTAL

NeuralForge fue desplegado en un clúster heterogéneo [8] de un grupo heterogéneo limitado a 30 nodos GPU. Medimos la latencia de enrutamiento de tareas y la huella de memoria frente a un trabajador Celery monolítico estándar ejecutando bucles secuenciales de PyTorch en el mismo proceso de Python. Llevamos a cabo un estudio de ablación desactivando temporalmente los límites de memoria del Invoker para observar la degradación del nodo anfitrión.

V. RESULTADOS Y DISCUSIÓN

NeuralForge logró una latencia de despacho de tareas de sub-milisegundos (0.8ms promedio) en los un grupo heterogéneo limitado a 30 nodos. El patrón Invoker-Executor mantuvo con éxito la estabilidad del anfitrión durante un período de entrenamiento continuo de 72 horas. En nuestro estudio de ablación, desactivar los límites de memoria de Docker en el Executor resultó en la eliminación del anfitrión por OOM en 4 horas debido a las fugas de memoria compartida del dataloader de PyTorch, demostrando la necesidad absoluta de la contenedorización efímera. El sistema de colas priorizadas redujo el tiempo de inactividad total de la GPU en un 40 %

al llenar los vacíos de baja prioridad con trabajos de ajuste secundarios. Los disparadores de transmisión de Watchtower permitieron actualizar la imagen del Executor en los un grupo heterogéneo limitado a 30 nodos sin tiempo de inactividad.

VI. DISPONIBILIDAD DE DATOS Y CÓDIGO

El sistema se publica bajo una Licencia Dual (PolyForm / AGPLv3). La infraestructura lista para producción puede desplegarse usando el repositorio `wyoloservice2_production` vía `docker-compose up -d`. Este trabajo es un módulo central de `wisrovi-suit` (<https://github.com/wisrovi/w-cli>).

VII. IMPACTO MÁS AMPLIO

La orquestación eficiente de GPUs mitiga directamente la huella de carbono [9] de la infraestructura de deep learning. La asignación dinámica de recursos de NeuralForge asegura la máxima utilización del hardware, disminuyendo significativamente el consumo de energía inactivo. Además, la capacidad de ejecución local completa garantiza la privacidad de los datos [10] (seguridad Shift-Left).

VIII. CONCLUSIÓN

NeuralForge presenta un cambio de paradigma en MLOps local para visión por computadora, demostrando que una arquitectura Invoker-Executor puede escalar de forma eficiente cargas de trabajo de HPO sin la sobrecarga de Kubernetes. Este framework proporciona un entorno optimizado y robusto para el entrenamiento iterativo de YOLO a escala empresarial.

IX. AGRADECIMIENTOS

Agradecemos al proyecto `wisrovi-suit` y a su comunidad por proporcionar las herramientas subyacentes que facilitaron la creación de este framework distribuido.

REFERENCIAS

- [1] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux journal*, vol. 2014, no. 239, p. 2, 2014.
- [2] X. Shi *et al.*, “Understanding out-of-memory errors in deep learning,” in *ISSTA*, 2021.
- [3] B. Momjian, *PostgreSQL: introduction and concepts*. Addison-Wesley New York, 2001, vol. 192.
- [4] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe *et al.*, “Accelerating the machine learning lifecycle with mlflow,” in *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018, pp. 39–45.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.
- [6] S. Ramirez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production,” URL: <https://fastapi.tiangolo.com>, 2020.
- [7] A. Sobolev, “Celery: Distributed task queue,” *Python project*, 2015.
- [8] Y. Li *et al.*, “Heterogeneous gpu scheduling for deep learning clusters,” in *INFOCOM*, 2020.
- [9] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [10] R. Shokri and V. Shmatikov, “Privacy-preserving deep learning,” in *CCS*, 2015.